República Bolivariana de Venezuela
Ministerio del Poder Popular para la Educación
Universidad Politécnica Territorial "José Antonio Anzoátegui"
PNF EN INFORMÁTICA – EXTENSIÓN GUANTA

INFORME DE REPASO DE LA PROGRAMACION II (F1)

Programación II

Profesor: Lennis Hunmer

Alumno: Ángel Cortesía: 32779999

9-11-2026

ÍNDICE

PORTADA 1

ÍNDICE 2

INTRODUCCIÓN 3

DESARROLLO 4

Fundamentos de la Programación Orientada a Objetos (POO) 4

Principios de la Programación Orientada a Objetos y Diagrama de Clases 4

Entidades Conceptuales Principales, Diagrama de objetos y atributos de clase y objeto 5-7

Métodos de cada clase 7-9

Relaciones y Multiplicidad 9

Diagrama UML (Ejemplo) 10

Especificadores 11

Constructores 11

Herencia y Clases Abstractas 11-12

Polimorfismo e Interfaces 12-13

Arquitectura Web 13

Diseño (HTML/CSS y Vistas Blade) 13-15

CONCLUSIÓN 16

INTRODUCCIÓN

El desarrollo del Sistema de Gestión Escolar para la U.E. "Juan Vicente González" se plantea como una solución informática orientada a optimizar y automatizar los procesos administrativos y académicos de la institución. El diseño de la propuesta se estructura bajo un enfoque modular y escalable que integra la Programación Orientada a Objetos y el patrón Modelo-Vista-Controlador (MVC) en Laravel. Este esquema arquitectónico garantiza una separación clara entre la interfaz de usuario, la lógica de negocio y la persistencia de datos en MySQL, facilitando la integridad referencial, la seguridad en los accesos por roles y la optimización de los flujos de información escolares.

DESARROLLO

Fundamentos POO:

La Programación Orientada a Objetos (POO) es un paradigma de programación, es decir, un modelo o un estilo de programación que nos da unas guías sobre cómo trabajar con él. Se basa en el concepto de clases y objetos:

Una clase es una plantilla. Define de manera genérica cómo van a ser los objetos de un determinado tipo.

Un objeto, como decíamos antes, podría ser un animal. Un animal tiene una edad, por lo que creamos un nuevo atributo de 'edad' y, además, puede envejecer, por lo que definimos un nuevo método.

Principios de la Programación Orientada a Objetos:

La encapsulación: contiene toda la información importante de un objeto dentro del mismo y solo expone la información seleccionada al mundo exterior.

La abstracción: es cuando el usuario interactúa solo con los atributos y métodos seleccionados de un objeto, utilizando herramientas simplificadas de alto nivel para acceder a un objeto complejo.

La herencia: define relaciones jerárquicas entre clases, de forma que atributos y métodos comunes puedan ser reutilizados.

El polimorfismo: consiste en diseñar objetos para compartir comportamientos, lo que nos permite procesar objetos de diferentes maneras.

Diagrama de Clases: Demuestra cómo se relacionan las entidades lógicas del sistema (Usuario, Estudiante, Docente, Nota) mediante atributos, métodos y claves relacionales que garantizan la integridad con la base de datos en MySQL.

Entidades Conceptuales Principales:

- Usuario: Clase base encargada de administrar la seguridad y el acceso al sistema mediante credenciales y roles.
- Estudiante: Entidad que representa al alumnado y sus datos de matriculación.
- Docente: Entidad responsable de las funciones académicas como la carga de notas y el control de asistencia.
- Nota: Entidad intermedia diseñada para registrar y estructurar el rendimiento académico por materias y lapsos.

Diagrama de Objetos (Estructura Dinámica): Representa una "fotografía" o estado del sistema en un momento específico de su ejecución, transformando los moldes conceptuales (clases) en instancias con datos reales y específicos.

Atributos de cada clase:

Clase: Usuario

Visibilidad	Atributo	Tipo de Dato	Observación
- (Privado)	cedula	Entero (int)	Clave Primaria (PK)
- (Privado)	nombre	Texto (string)	Nombre completo
- (Privado)	contraseña	Texto (string)	Credencial de acceso
- (Privado)	rol	Texto (string)	'admin', 'docente', 'estudiante', 'representante'

Clase: Estudiante

Visibilidad	Atributo	Tipo de Dato	Observación
- (Privado)	id_estudiante	Entero (int)	Identificador del alumno
- (Privado)	cedula	Entero (int)	Cédula del estudiante
- (Privado)	grado_seccion	Texto (string)	Ejemplo: "5to Año - Sección A"
- (Privado)	id_representante	Entero (int)	Enlace al representante

Clase: Docente

Visibilidad	Atributo	Tipo de Dato	Observación
- (Privado)	id_docente	Entero (int)	Identificador del profesor
- (Privado)	cedula	Entero (int)	Cédula del docente

Visibilidad	Atributo	Tipo de Dato	Observación
- (Privado)	especialidad	Texto (string)	Materia asignada (ej. "Matemáticas")

4. Clase: Nota

Visibilidad	Atributo	Tipo de Dato	Observación
- (Privado)	id_nota	Entero (int)	Identificador del registro
- (Privado)	id_estudiante	Entero (int)	Pertenece a un estudiante
- (Privado)	id_docente	Entero (int)	Registrado por un docente
- (Privado)	materia	Texto (string)	Asignatura evaluada
- (Privado)	lapso	Entero (int)	Valores: 1, 2 o 3

Visibilidad	Atributo	Tipo de Dato	Observación
- (Privado)	calificacion	Decimal (float)	Nota del lapso (ej. 18.0)

Métodos de cada clase:

1. Clase: Usuario

Visibilidad	Método	Descripción / Función
+ (Público)	iniciarSesión()	Valida las credenciales de acceso en el sistema.
+ (Público)	cerrarSesión()	Destruye la sesión activa de forma segura.

2. Clase: Estudiante

Visibilidad	Método	Descripción / Función
+ (Público)	consultarBoleta()	Permite al alumno visualizar sus notas en pantalla.

3. Clase: Docente

Visibilidad	Método	Descripción / Función
+ (Público)	cargarNota()	Registra las calificaciones de un estudiante en la base de datos.
+ (Público)	registrarAsistencia()	Guarda el control de asistencia diario.

4. Clase: Nota

En esta clase no se define métodos explícitos, ya que funciona fundamentalmente como una entidad de almacenamiento y registro de datos.

Relaciones y Multiplicidad:

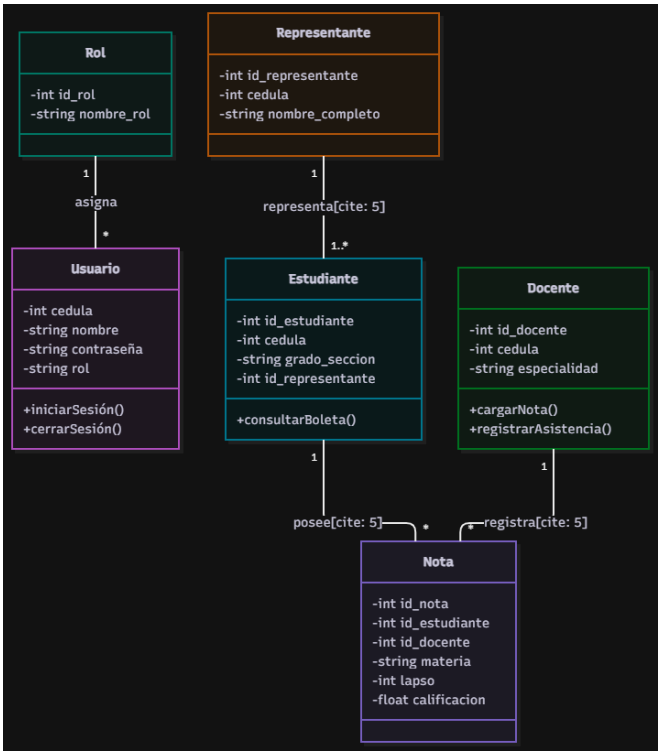
Usuario a Roles (1 a Muchos): Un Usuario tiene un único rol, pero un Rol puede pertenecer a muchos usuarios en el sistema.

Docente a Nota (1 a Muchos): Un Docente puede registrar muchas Notas.

Estudiante a Nota (1 a Muchos): Un Estudiante posee muchas Notas asociadas en su boleta.

Representante a Estudiante (1 a Uno o Varios): Un Representante puede tener uno o varios Estudiantes a su cargo.

Diagrama UML (EJEMPLO):



La definición de clases y objetos estructura de forma eficiente el desarrollo del software. El modelado de clases garantiza la integridad de la base de datos relacional en MySQL mediante claves foráneas bien definidas. A su vez, la abstracción en diagramas de objetos simula la interacción real del sistema, validando que el enrutamiento y la asignación de notas respondan con precisión a las necesidades de la institución

Especificadores: Son palabras clave que definen la visibilidad y seguridad de los atributos y métodos de una clase, es decir, quiénes pueden verlos o usarlos.

Público (+ / public): Accesible desde cualquier parte del programa, sin restricciones

Privado (- / private): Solo se puede acceder a ellos dentro de la misma clase donde fueron declarados. Nadie de afuera puede tocarlos directamente (ideal para proteger datos como contraseñas o cédulas).

Protegido (# / protected): Accesible dentro de la misma clase y por sus clases hijas (útil cuando trabajamos con herencia).

Constructores: Un constructor es un método especial que se ejecuta automáticamente cuando creas un objeto a partir de una clase (por ejemplo, cuando usas new Estudiante()).

Para qué sirve: Su función principal es inicializar los atributos del objeto con valores por defecto o con los datos que le pases al momento de crearlo.

Característica: Se llama exactamente igual que la clase y no devuelve ningún tipo de dato (ni siquiera void).

Herencia: Es el mecanismo que permite que una clase (llamada Clase Hija o subclase) herede los atributos y métodos de otra clase (llamada Clase Padre o superclase).

Para qué sirve: Evita repetir código. Si tienes varias clases con cosas en común (por ejemplo, Estudiante y Docente comparten atributos como cedula y nombre), puedes crear una clase padre llamada Persona y hacer que los demás hereden de ella.

En código: Se usa la palabra clave extends (en lenguajes como Java o PHP).

Clases Abstractas: Son clases especiales que no se pueden instanciar directamente (es decir, no puedes crear un objeto usando new de una clase abstracta).

Para qué sirve: Funcionan como una plantilla o molde genérico para otras clases.

Métodos abstractos: Una clase abstracta puede tener métodos sin código (sin cuerpo). Las clases hijas que hereden de ella están obligadas a implementar y programar esos métodos a su manera.

Polimorfismo: La palabra significa literalmente "muchas formas". En programación, se refiere a la capacidad de que un mismo método o acción se comporte de manera diferente según el objeto que lo esté ejecutando.

Ejemplo clásico: Imagina un método llamado hablar(). Si lo ejecuta un objeto de la clase Perro, dirá *"Guau"*, si lo ejecuta un objeto de la clase Gato, dirá *"Miau"*. El nombre del método es el mismo, pero el resultado cambia según quién lo invoque (esto se logra mediante la sobrescritura de métodos).

Interfaces: Las interfaces funcionan como contratos o plantillas que obligan a las clases a implementar métodos específicos sin definir la lógica interna, garantizando un estándar de comportamiento común en toda la aplicación. A diferencia de las clases abstractas, una interfaz no contiene código heredable ni atributos de estado, sino únicamente las firmas de los métodos que deben ser programados obligatoriamente por cualquier clase que decida implementarla. Por ejemplo, se puede definir una interfaz llamada Autenticable que obligue a clases como Estudiante y Docente a implementar el método validarAcceso, asegurando que ambos perfiles respondan al mecanismo de seguridad del sistema de manera uniforme. Asimismo, se puede aplicar una interfaz de reportes para que las clases encargadas de procesar notas o asistencias cumplan con la obligación de generar la exportación de datos requerida por la institución educativa, manteniendo así un código desacoplado, limpio y fácil de escalar en el desarrollo web.

Arquitectura Web: La Arquitectura Web se fundamenta en un modelo cliente-servidor estructurado bajo el patrón Modelo-Vista-Controlador. En el cliente o frontend, se emplean interfaces desarrolladas con HTML, CSS y JavaScript para capturar los datos de los usuarios y mostrar las vistas de navegación de acuerdo con los roles asignados. El servidor o backend, desarrollado en PHP con el framework Laravel, procesa las solicitudes a través de un enrutamiento seguro que gestiona la lógica de negocio, como la validación de credenciales en el inicio de sesión o el registro de calificaciones. Por su parte, la capa de datos interactúa de forma directa con la base de datos relacional MySQL, garantizando la integridad referencial y la persistencia de la información. La comunicación entre el navegador y el servidor se realiza mediante peticiones HTTP y HTTPs, asegurando una transferencia eficiente de datos y permitiendo que la aplicación web funcione de manera escalable, modular y alineada a las necesidades operativas de la institución educativa.

Diseño (HTML/CSS): Para la capa de presentación y la interfaz de usuario, se emplearon HTML5, CSS3 y diseño responsivo integrados directamente con el motor de plantillas Blade de Laravel. Este enfoque permite construir componentes visuales limpios, modulares y adaptables a diferentes dispositivos, facilitando una experiencia de usuario fluida e intuitiva según el rol de acceso (administrador, docente, estudiante o representante). Las vistas se organizan jerárquicamente en el directorio resources/views/, separando la estructura global de la aplicación (layouts/app.blade.php) de las interfaces específicas de

gestión (students/, grades/) y la portada (welcome.blade.php), lo que garantiza la coherencia estética, la reutilización de código y un mantenimiento eficiente de la interfaz web.

Configuración General y Entorno de Trabajo:

Estructura del Proyecto (.editorconfig, .env.example, composer.json): Contiene los archivos de configuración iniciales para estandarizar el código y las dependencias del framework Laravel, gestionando las librerías necesarias y las variables de entorno para la conexión con la base de datos.

Gestión de Rutas y Controladores (Backend) Rutas (routes/web.php, routes/api.php): Establecen los puntos de entrada de la aplicación web, separando las rutas de navegación de usuarios y las peticiones internas del sistema.

Controladores (app/Http/Controllers/):

StudentController.php: Diseñado para gestionar la lógica de control referente a los estudiantes (registro, consulta y actualización de datos).

GradeController.php: Encaminado a manejar la lógica de negocio para la carga, asignación y visualización de calificaciones dentro del sistema escolar.

Modelos de Datos y Migraciones (Base de Datos)

Modelos (app/Models/):

User.php: Representa la entidad de usuarios generales y autenticación.

Student.php: Representa la entidad de los estudiantes interconectada con la base de datos.

Grade.php: Representa el modelo para el control de notas y calificaciones.

Migraciones (database/migrations/): Contienen los scripts con estructura de fechas (como las tablas creadas para estudiantes y calificaciones) que permiten estructurar las tablas relacionales en MySQL de forma automatizada mediante Artisan.

Vistas y Interfaz de Usuario (Frontend)

Plantillas Blade (resources/views/):

layouts/app.blade.php: Define la estructura visual base o diseño global compartido de la aplicación.

students/ y grades/: Contienen las vistas organizadas (como formularios de creación, edición e índices de listado) para que los usuarios interactúen con la gestión de alumnos y notas.

welcome.blade.php: Vista de la página de bienvenida o portada del sistema web.

El diseño propuesto para el Sistema de Gestión Escolar de la U.E. "Juan Vicente González" establece una base técnica sólida y estructurada para automatizar los procesos académicos e institucionales. A través de la integración de la Programación Orientada a Objetos, el uso de migraciones y modelos en Laravel, y un esquema relacional optimizado en MySQL, la arquitectura garantiza la integridad de los datos, la seguridad en la gestión de usuarios y la escalabilidad del sistema. Esta propuesta cumple rigurosamente con los requerimientos técnicos exigidos en la formación académica, ofreciendo una solución moderna, limpia y funcional adaptada a las necesidades reales de la institución educativa.

CONCLUSIÓN

El desarrollo del Sistema de Gestión Escolar para la U.E. "Juan Vicente González" consolida de manera integral todos los conocimientos técnicos y teóricos adquiridos a lo largo del trayecto académico. A través de la aplicación de la Programación Orientada a Objetos, el dominio de los especificadores de acceso, las herencias, clases abstractas, polimorfismo e interfaces, se logró estructurar un código robusto y mantenible. Asimismo, la implementación del patrón Modelo-Vista-Controlador (MVC) utilizando el framework Laravel en conjunto con una base de datos relacional en MySQL garantiza una arquitectura web escalable, segura y eficiente. Este proyecto no solo cumple con los requerimientos metodológicos exigidos en la formación universitaria, sino que presenta una solución informática real, limpia y funcional capaz de optimizar los procesos administrativos y académicos de la institución.